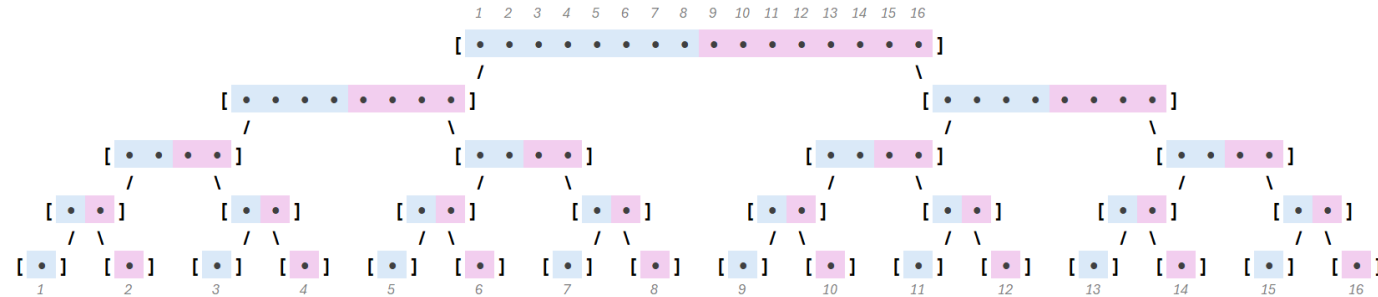


ALGORITHME DE TRI FUSION

Un algorithme basé sur : Diviser pour régner



RÉALISATION DE LA FONCTION DE TRI FUSION

Codage & tests

INTRODUCTION

- **Encore un autre algorithme de tri ?**

- Nous avons déjà vu les algorithmes de tri suivants :
 - Tri par sélection
 - Tri par insertion
 - Tri à bulles
- Cependant bien que l'étude de ces algorithmes étaient très formateurs, malheureusement, ces algorithmes sont peu efficaces. C'est-à-dire qu'ils sont relativement lents par rapport à d'autres algorithmes de tri et notamment par rapport au tri fusion que nous voyons maintenant.

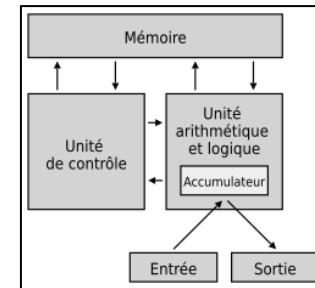
INTRODUCTION

- **La stratégie utilisée :**

- L'algorithme de tri fusion va exploiter la stratégie « **diviser pour régner** ». Nous avons déjà appliqué cette stratégie pour l'algorithme de recherche par dichotomie.

- **Origine :**

- John von Neumann était un éminent scientifique et a fortement contribué au développement de l'informatique, notamment en participant à la conception de l'**ENIAC** et de son processeur dont l'architecture dite *architecture von Neumann* est encore aujourd'hui utilisée dans de nombreux processeurs.
- Il inventa l'algorithme de tri fusion dans le cadre de ses développements avec les premiers ordinateurs.



Architecture von Neumann
des processeurs



John von Neumann dans les
années 1940

PRINCIPE DE BASE DE L'ALGORITHME

- **L'algorithme applique la stratégie diviser pour régner, ce qui revient à décomposer l'algorithme en 3 parties :**



- **Diviser** : On découpe la liste initiale en 2 sous-liste, qui sont elles-mêmes redécoupée en 2 jusqu'à atteindre une sous-liste ne comportant qu'une seule valeur.



- **Régner** : Une fois les listes découpées en liste d'une valeur, le sous-problème est implicitement résolu



- **Combiner** : Il faut recombinaison les sous-listes pour obtenir des listes de valeurs strictement croissante.

IMPLÉMENTATION DE L'ALGORITHME

- **Description de la démarche :**

- La décomposition en sous-listes sera réalisée par un algorithme **récuratif**. Nous étudierons cette partie ultérieurement.
- Pour le moment nous allons nous concentrer sur l'assemblage des sous-listes. Pour ce faire nous allons développer une fonction que nous nommerons **fusion()**.

IMPLÉMENTATION DE L'ALGORITHME

- **Travail à faire :**

- Ouvrir **Thonny** et créer un fichier **TriFusion.py**
- Procédons à l'implémentation de la fonction **fusion()** :
 - La fonction prend en argument :
 - 2 listes d'entiers triées dans l'ordre croissant
 - La fonction retourne :
 - Une liste contenant l'ensemble des valeurs (type entier) des 2 listes passées en argument
 - Les valeurs sont toutes classées dans l'ordre croissant.
- Copier l'entête suivante dans votre fichier **TriFusion.py** :

```
def fusion(l1: [int], l2: [int]) -> [int]:  
    """Prend en argument 2 listes d'entiers l1 et l2  
    dont les valeurs sont classées dans l'ordre croissant.  
    Retourne une liste contenant l'ensemble des valeurs  
    de l1 et l2 classées dans l'ordre croissant."""
```

IMPLÉMENTATION DE L'ALGORITHME

- **Travail à faire :**

- Nous vous proposons la trame suivante que vous allez recopier et compléter :

```
l_fusion = []
# Cas où les listes sont vides :
if len(l1) == 0 and len(l2) == 0:
    return l_fusion

i1 = 0
i2 = 0
# Copie par récupération de la valeur la plus faible dans l1 et l2 :
while i1 < len(l1) and i2 < len(l2):
    if l1[i1] < l2[i2]:
        ...
    else:
        l_fusion.append(l2[i2])
        i2 += 1

# Copie de la/des valeur(s) restant(es) dans la liste qui n'a pas été entièrement parcourue :
if i1 < len(l1):
    l_fusion += l1[i1:]
else:
    ...

return l_fusion
```

- Vous trouverez à la page suivante des tests unitaires permettant de valider votre code.

IMPLÉMENTATION DE L'ALGORITHME

- **Travail à faire :**

- Vous pourrez valider le bon fonctionnement de votre fonction avec les tests suivants.
- Remarque : Le code peut-être ici directement copié->collé

```
assert fusion([1, 3, 5, 6, 9, 10, 11], [2, 4, 7, 8]) == [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11], "Liste de sortie non triées !"
assert fusion([1, 3, 5, 6, 9, 10, 11], []) == [1, 3, 5, 6, 9, 10, 11], "Liste de sortie non triées !"
assert fusion([], []) == []
```

IMPLÉMENTATION DE L'ALGORITHME

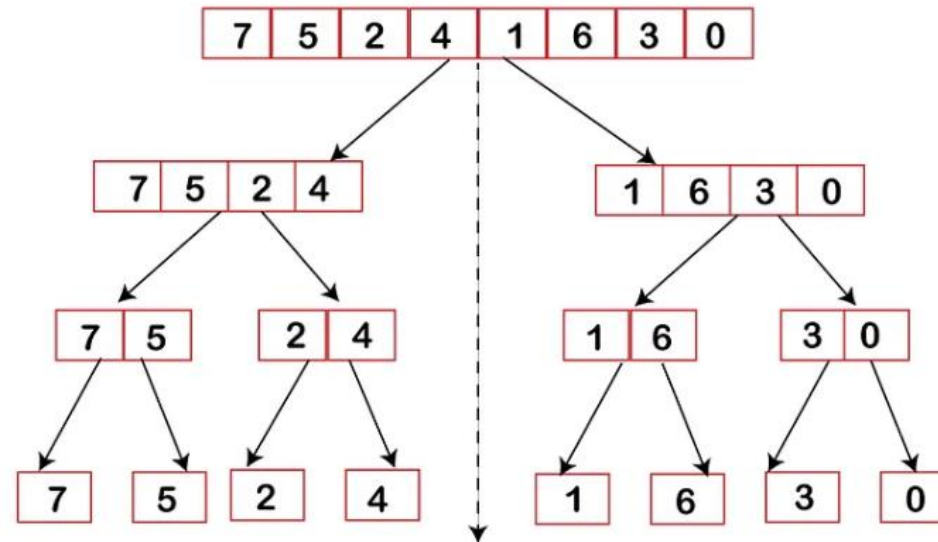
- **L'algorithme récursif :**

- Maintenant que nous avons la fonction de fusion, nous allons pouvoir nous concentrer sur la fonction récursive qui aura pour charge :
 - Si la liste donnée en argument a une taille strictement supérieure à 1, alors :
 - Diviser la liste donnée en argument en 2 sous-listes.
 - Appeler récursivement 2 fois la fonction avec chaque sous-liste en argument.

IMPLÉMENTATION DE L'ALGORITHME

- **L'algorithme récursif :**

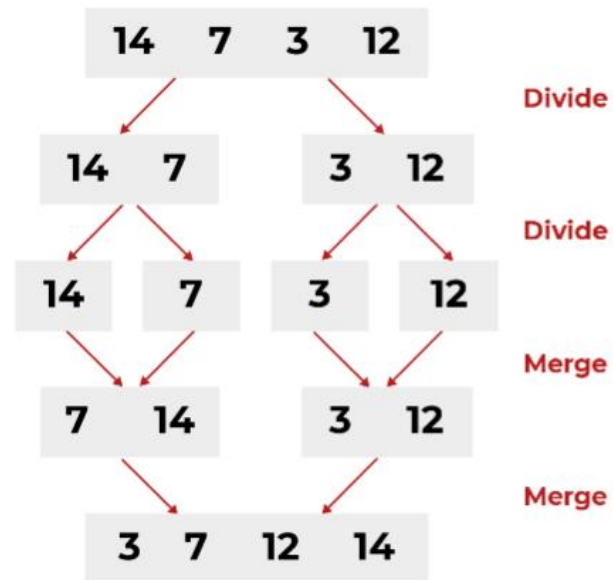
- On peut représenter les appels récursifs avec le graphe suivant pour illustrer son fonctionnement.



IMPLÉMENTATION DE L'ALGORITHME

- **L'algorithme récursif :**

- Et voici le déroulement complet du tri fusion sur un exemple :



IMPLÉMENTATION DE L'ALGORITHME

- **Travail à faire :**

- Copier et compléter la fonction **tri_fusion()** :

```
def tri_fusion(l: [int]) -> [int]:  
    """Prend une liste d'entiers en argument.  
    Retourne une liste avec l'ensemble des valeurs  
    triées dans l'ordre croissant."""  
  
    if len(l) <= 1:  
        ...  
    else:  
        l1 = tri_fusion(l[: len(l) // 2])  
        l2 = ...  
        return ...
```

IMPLÉMENTATION DE L'ALGORITHME

- **Travail à faire :**

- Vous pourrez réaliser les tests unitaires suivants :

```
assert tri_fusion([5, 8, 2, 1, 3, 4, 7, 6]) == [1, 2, 3, 4, 5, 6, 7, 8], "Liste de sortie non triées !«  
assert tri_fusion([5]) == [5]  
assert tri_fusion([]) == []
```

EVALUATION DE LA PERFORMANCE

Tracé de la courbe $t(n)$

EVALUATION DE LA PERFORMANCE

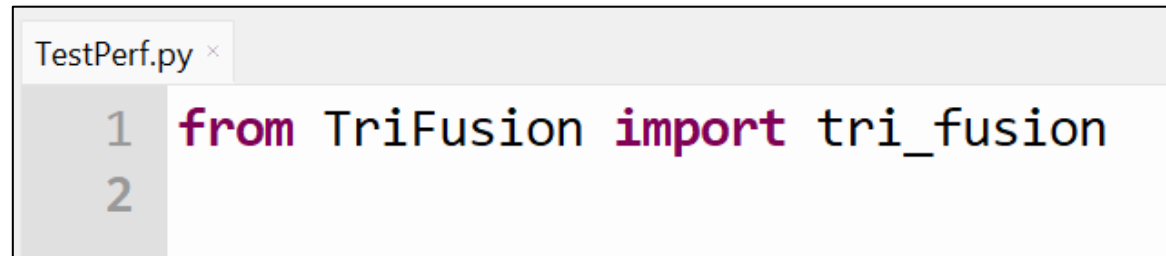
- **Présentation de la démarche :**

- L'algorithme de tri fusion est un algorithme relative rapide, c'est-à-dire que la durée d'exécution du tri évoluera relativement peu avec l'augmentation des données à trier.
- Nous vous proposons de vérifier cette propriété en traçant avec la bibliothèque **Matplotlib** une courbe qui représente le temps de la fonction de tri en fonction du nombre de valeurs n à trier.
- Pour arriver à cela nous allons procéder en 3 étapes :
 - Génération de k listes d'entiers non triés
 - Exécution et mesure du temps T pour chaque liste.
 - Tracé de la courbe à partir des tests précédents.

EVALUATION DE LA PERFORMANCE

- **Import des dépendances :**

- Avant de commencer, ouvrir un nouveau fichier dans **Thonny** que vous nommerez **TestPerf.py**
- Puis importer la fonction **tri_fusion** du fichier **TriFusion.py**



The screenshot shows a code editor window titled 'TestPerf.py'. The editor contains two lines of Python code. Line 1 is 'from TriFusion import tri_fusion'. Line 2 is empty. The code is written in a monospaced font with syntax highlighting: 'from' is purple, 'TriFusion' is blue, 'import' is purple, and 'tri_fusion' is blue.

```
TestPerf.py ×  
1 from TriFusion import tri_fusion  
2
```

EVALUATION DE LA PERFORMANCE

- **Import des dépendances :**

- Nous allons ensuite importer :
 - **random** : Pour générer des listes de tests avec des valeurs non triées.
 - **time** : Pour chronométrer les durées d'exécution de notre fonction de tri.
 - **matplotlib** : Pour tracer la courbe $T(n)$

```
1 from TriFusion import tri_fusion
2 from random import shuffle
3 from time import perf_counter
4 import matplotlib.pyplot as plt
```

EVALUATION DE LA PERFORMANCE

- **Génération des listes de tests :**

- Nous allons maintenant générer des listes aléatoires d'entiers.
- La taille des listes est multipliée par 5 à chaque génération.

- **Rajouter le code suivant à votre programme :**

```
7 # Génération d'un jeu de test :
8 # -----
9 k = 12
10 l_tests = []
11
12 for i in range(k):
13     liste_test = [ j for j in range(int(10**(i / 2)))]
14     shuffle(liste_test)
15     l_tests.append(liste_test)
16
```

EVALUATION DE LA PERFORMANCE

- **Exécution des tests :**

- Nous allons maintenant pour chaque liste de test exécuter la fonction de tri fusion.
- Nous allons relever la date avant et après l'exécution afin de calculer la durée d'exécution pour l'enregistrer dans la liste **durees**.

```
18 # Exécution chronométrée des tests avec consignation des résultats :
19 # -----
20 nb_val = []
21 durees = []
22
23 for liste_test in l_tests:
24     t_deb = perf_counter()
25     tri_fusion(liste_test)
26     t_fin = perf_counter()
27
28     nb_val.append(len(liste_test))
29     durees.append(t_fin - t_deb)
30
```

EVALUATION DE LA PERFORMANCE

- **Génération de la courbe :**

- Nous avons maintenant les données nécessaires pour tracer notre courbe $T(n)$.

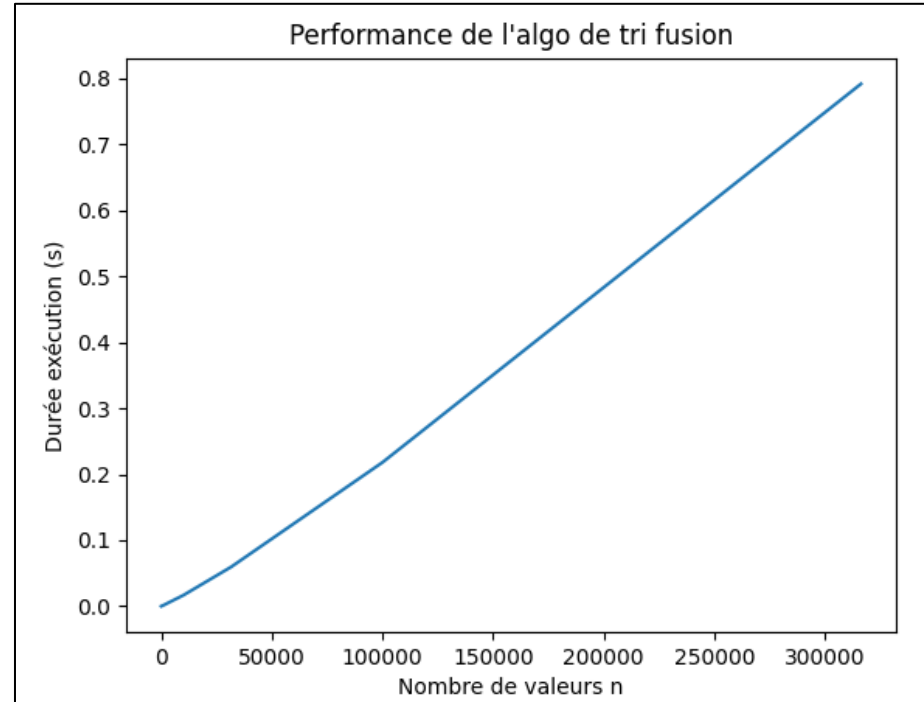
- **Rajouter le code à votre programme :**

```
32 # Tracé de la courbe :  
33 # -----  
34 plt.title("Performance de l'algo de tri fusion")  
35 plt.xlabel("Nombre de valeurs n")  
36 plt.ylabel("Durée exécution (s)")  
37 plt.plot(nb_val, durees)  
38 plt.show()  
39
```

EVALUATION DE LA PERFORMANCE

- **Exécution :**

- Exécuter votre programme, vous devriez obtenir la courbe suivante :



Fin